Final Project Report

Moviera

Team Name	Krisztina Mészáros
Delivery Date	13/06/2026
Course	Mobile Application Development

Team Members

Name and Surname	N. Matricola
Krisztina Mészáros	311117

Table of Contents

Contents

1. Introduction

1.1 Project Context and Motivations

In the modern digital entertainment landscape, users are faced with an overwhelming abundance of content across numerous streaming platforms. This phenomenon often leads to choice overload and *decision fatigue*.

A common real-world problem occurs when individuals sit down to watch a movie after a long day, only to spend a significant amount of time endlessly scrolling through catalogs without being able to choose what to watch.

The motivation behind this project stems from the need to eliminate this friction. While several movie-tracking applications already exist on the market, they are often bloated with excessive information, complex cross-references, cast lists, and endless reviews. This app was born from the idea that a movie-saving tool should be lightweight and hyper-focused. It addresses the common scenario where a user receives a quick movie recommendation during the day and needs an immediate, frictionless way to store that title for later. By providing a clean and rapid solution, the project aims to solve the problem of evening indecision by shifting the discovery phase away from the TV screen into a simple, personal shortlist.

1.2 General Purpose of the Application

The application, named **Moviera**, is a minimalist mobile movie-tracker powered by the external *The Movie Database (TMDB)*[1] API. Its primary purpose is to allow users to instantly search for movies, view essential details, and save them into a personal, offline-accessible watch list. Additionally, it enables users to seamlessly toggle the "*seen*" status of their saved movies to keep track of their viewing history.

The core value **Moviera** brings to its users lies in its simplicity and efficiency. Instead of encouraging long, passive browsing sessions within the application, the app is intentionally designed for short, frequent interactions, allowing a user to look up and save a movie in a matter of seconds. By stripping away unnecessary clutter, it serves as a reliable personal digital notepad that directly saves time and enhances the daily entertainment experience.

1.3 Target and Needs

The target audience for **Moviera** encompasses a wide range of everyday users, from university students to busy professionals, who enjoy watching movies but want to organize their viewing habits without digital noise.

Specifically, the application addresses the following distinct user needs:

- **Frictionless Capture**: The need to immediately look up and log a movie recommendation before the title is forgotten.
- **Curated Personal Space**: The need for a dedicated, clutter-free repository of specific movies the user actually intends to watch, separating personal choices from algorithmic streaming recommendations.
- **Offline Accessibility**: The need to view and manage one's own saved watchlist anytime, anywhere, even without an active internet connection.
- **Viewing Progress Tracking**: A simple way to mark content as completed, providing a clear overview of what has been watched versus what is left to enjoy.

[1] https://www.themoviedb.org/

2. Domain Selection and Analysis

2.1 Description of the Application Domain

The application domain for **Moviera** falls squarely into the *Business Productivity* and *Personal Media Management* sector. This domain focuses on providing users with software tools designed to optimize time, streamline personal data organization, and eliminate unnecessary friction in daily decision-making processes. Specifically, in the context of mobile entertainment, this domain covers the curation, indexing, and personal archiving of digital media metadata (such as titles, ratings, and release years) retrieved asynchronously from cloud-based external providers. Key features of this application domain includes: lightweight database caching, cross-platform UI synchronization, and precise data filtering based on explicit user actions.

2.2 Reasons for the Choice

The selection of this domain was guided by multiple strategic factors aligned with our current software development criteria:

- **User-Centric Innovation**: Traditional entertainment management platforms focus heavily on community interaction, deep cross-referencing, and long exploration phases. By entering this domain, the project attempts to innovate through subversion, focusing exclusively on extreme simplicity and productivity.
- **Resource Optimization**: Given the finite timeframe and development constraints of the current academic sprint cycle, this domain allows the team to deliver high user value without getting bogged down by overly complex custom backend requirements.
- **Technical Relevance**: This domain provides a perfect testbed for demonstrating core mobile engineering competencies, including REST API consumption, persistent local key-value caching, and reactive state synchronization across declarative user interfaces.

2.3 Analysis of Problems and Opportunities

An analysis of the chosen domain reveals a distinct set of challenges and clear avenues for application growth:

The Problem (*Choice Overload*): Digital streaming availability has triggered a widespread UX issue known as “choice paralysis” or “decision fatigue.” Users spend a disproportionate amount of time looking at content grids rather than enjoying the actual media. Existing tracking apps worsen this by flooding the interface with secondary data (cast bios, reviews, social feeds) that distracts from the core task

The Opportunity (*The “Digital Notepad” Approach*): There is a clear market gap for a utility that handles movie logging as a rapid, functional checklist rather than an immersive entertainment platform. **Moviera** capitalizes on this opportunity by filtering out non-essential data and leveraging a reliable third-party API (*TMDB*) to provide an incredibly fast, highly readable personal watchlist that acts as an immediate remedy for evening indecision.

2.4 SWOT Analysis

Strengths

- **Clear and narrow project scope**: Focusing strictly on a minimalist approach ensures a realistic development cycle and predictable deliverables.
- **High technical feasibility**: Utilizing the stable, comprehensive, and well-documented *TMDB API* minimizes backend development complexity.
- **Cross-platform native performance**: Building the application on *Flutter* ensures high frame rates and a unified, robust codebase for rapid deployment.
- **Robust offline operation**: Local data caching guarantees that saved content remains accessible even under poor network conditions.

Weaknesses

- **Third-party data dependency**: The core functionality of the search system relies completely on the availability and data quality of the external *TMDB API*.
- **Lack of advanced recommendation systems**: The app relies entirely on explicit user saving habits, missing automated algorithmic discovery tools.
- **Limited scope appeal**: Users looking for deep social integration, extensive cast reviews, or community discussions might find the app too sparse.

Opportunities

- **Targeting niche minimalism**: Capturing a growing demographic of users who actively seek minimalist, distraction-free software utilities.
- **Incremental feature scaling**: Future potential to add simple, low-overhead custom alerts (e.g., local push notifications when a saved movie becomes available to stream).
- **Frictionless onboarding**: By prioritizing speed, the app can integrate fast local caching mechanisms, minimizing standard application setup barriers.

Threats

- **Saturated core market**: The personal media tracking space is highly competitive, filled with large, deeply entrenched applications like IMDb, Letterboxd, or Todoist.
- **API policy changes**: Unannounced structural updates, bandwidth throttling, or monetization shifts by the TMDB platform could impact network service layers.
- **User retention boundaries**: The deliberate design choice to minimize user session length within the app could inadvertently lead to lower long-term engagement metrics if users forget the app is installed.

3. Project Objectives

3.1 General Objectives

The high-level goal of **Moviera** is to create a functional, production-ready, and highly polished mobile application that counters the pervasive issue of “choice paralysis” in digital media consumption. The project aims to deliver a specialized utility that replaces long, aimless scrolling sessions with a fast, zero-clutter “digital notepad” for movie tracking.

From an academic perspective, the objective is to apply agile software engineering methodologies, progressing from initial domain analysis to structural prototyping and iterative development, resulting in a cleanly architected cross-platform application. Ultimately, the project intends to prove that stripping away non-essential features can significantly enhance user efficiency and satisfaction within the personal organization space.

3.2 Functional Objectives

To achieve the general goals of the project, the application must fulfill the following specific and measurable key features:

- **Instant cloud-driven search**: Users must be able to asynchronously search for any movie title in real-time, fetching precise metadata (title, overview, release year, rating, and poster artwork) directly from the external TMDB API.
- **Persistent local watchlist**: Users must be able to save any discovered movie into a personal local watchlist with a single tap. This repository must remain fully accessible and functional without an active internet connection.
- **Viewing history tracking**: Users must have the ability to toggle a “*seen*” status for any movie in their personal list, allowing them to instantly filter and manage their completed content versus their remaining watchlist.

3.3 Technical and Innovative Objectives

The technical and innovative execution of the project is defined by the following modern development approaches:

- **Declarative cross-platform UI with reactive state management**: The application will be engineered using Flutter and Dart, utilizing a strict Model-View-ViewModel (MVVM) pattern combined with the Provider package. This ensures a decoupled codebase where business logic is separated from UI rendering, and state updates trigger precise widget rebuilds instead of costly full-screen refreshes.
- **Hyper-minimalist UX (*Information filtering*)**: The application innovates through purposeful constraints. Instead of building massive data structures with endless cross-references, the codebase and UI are explicitly optimized for interactions lasting less than 30 seconds, prioritizing raw speed and layout clarity over feature bloat.

3.4 Crazy Eights

During the initial design phase, a “Crazy Eights” brainstorming session was conducted to quickly explore various innovative layouts and feature variations under strict time pressure. The session generated eight distinct concepts before arriving at the final structural design:

1. **Classic grid *view***: A standard layout displaying movies as large poster grids with titles underneath, heavily prioritizing visual design over immediate information.
2. **Swipe-to-save interface**: A Tinder-like experimental interface where trending movies pop up as cards, and users swipe right to save to their watchlist or left to dismiss.
3. **Minimal text list**: A radical minimalist interface displaying only typography (titles and years) on a dark screen, completely removing images to ensure maximum loading speeds.

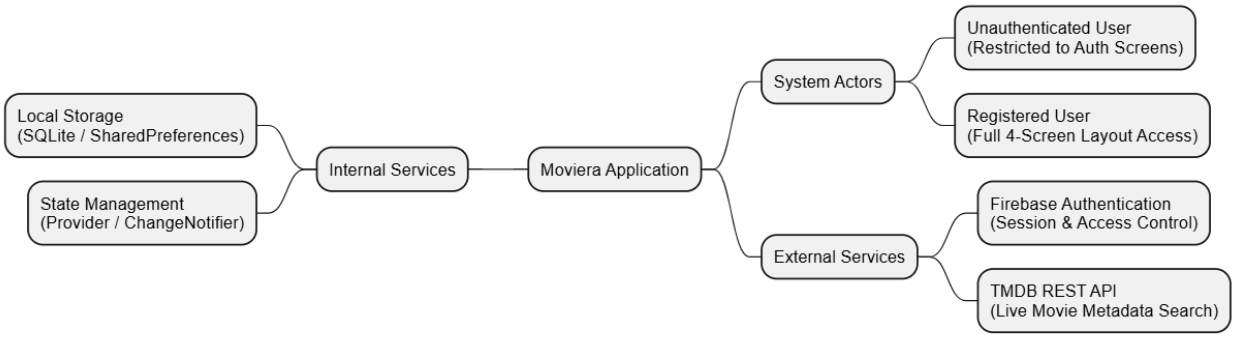
4. **Evening roulette button**: A single-button layout on the home screen that randomly selects one movie from the user's saved watchlist to instantly solve *decision fatigue*.
5. **Timeline viewing history**: A screen layout structured as a vertical timeline, showing movies in the order the user marked them as “seen” over the past months.
6. **Detailed overlay card**: A screen design where clicking a movie does not navigate to a new page but instead slides up a minimalist bottom sheet containing only the overview and a toggle button.
7. **The offline-first dashboard**: A layout that immediately loads local stored data on app launch, placing a small search icon at the top to highlight that the app is primarily a local repository.
8. **Minimalist layout (the selected concept):** A streamlined architecture designed explicitly for rapid user interaction. It features a bottom navigation bar to switch between the local saved watchlist (*Home*) and the live API search screen.

4. System Analysis

4.1 Mind Map of Actors and Services

The mind map below illustrates the architectural ecosystem of **Moviera**. It visualizes how the two identified user roles (Unauthenticated and Registered) interact with the core infrastructure of the application.

The system is structurally split between external cloud services, specifically Firebase Authentication for access control and the *TMDB API* for live data retrieval, and local on-device services, such as the persistent local database and the Provider state management layer that ensures seamless data synchronization.



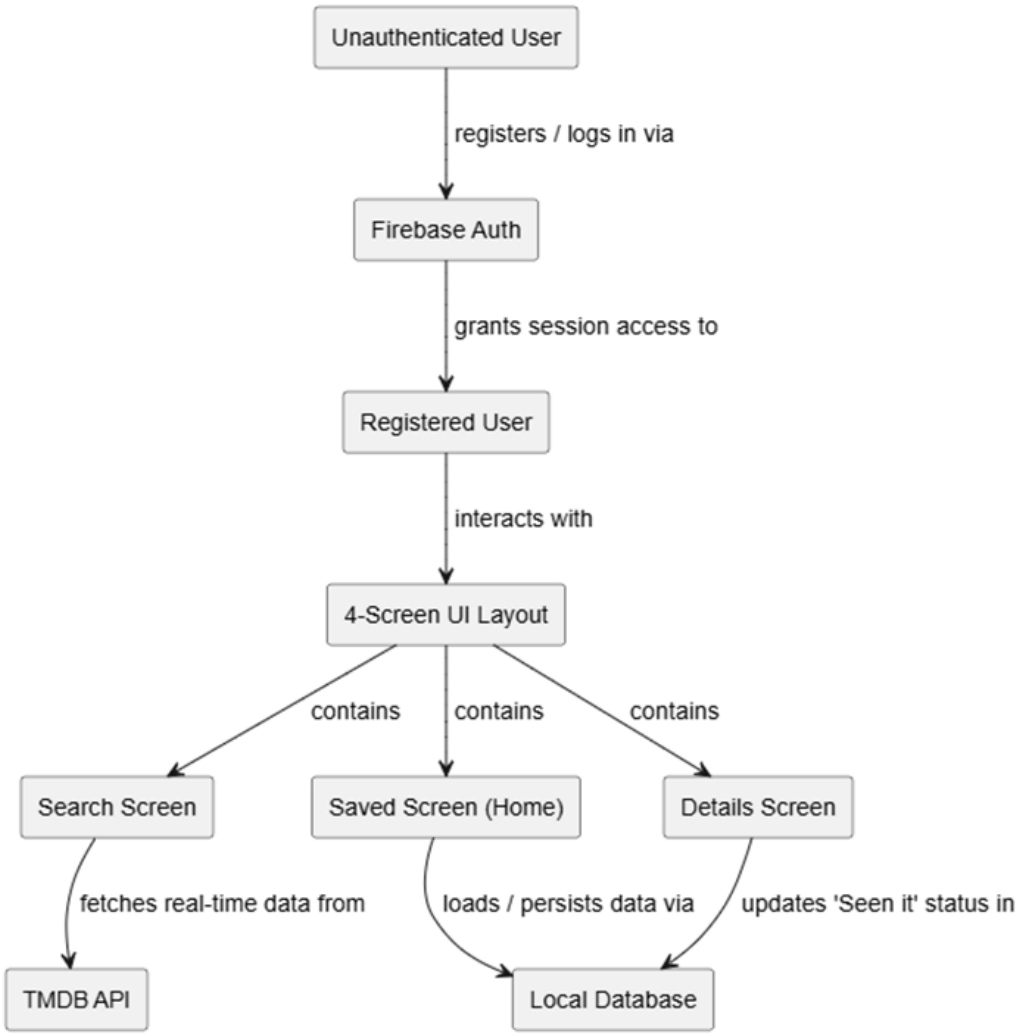
4.2 Description of the Actors and Their Roles

Actor	Role Description
Unauthenticated user	A guest user who has just launched the application and does not have an active session. Following a strict *gatekeeper* navigation pattern, this actor is completely restricted to the Login and Registration screens. They cannot bypass these screens, see the bottom navigation bar, or access any application features (such as searching or viewing the watchlist) until they successfully authenticate via *Firebase*.
Registered user	An authenticated end-user who has successfully logged in via Firebase Authentication. Once authenticated, the full 4-screen layout becomes accessible. This actor can look up new movies using the live *TMDB API*, manage their offline personal watchlist, toggle the "*Seen it*" status on the movie detail screen, and view the static *About* page.
Administrator	Given the minimalist, client-side nature of **Moviera**, there are no features that require content moderation, global user management, or central data overrides within the app. Therefore, an administrative account or role is omitted from the system architecture to maintain a lightweight, streamlined codebase.

4.3 Concept Map

The concept map below delineates the relational architecture and data flows within the **Moviera** application. It maps out the strict boundary between unauthenticated sessions and registered user spaces via *Firebase Authentication*.

Furthermore, the diagram clarifies the separation of data responsibilities: live search actions are strictly coupled with the cloud-based *TMDB API*, whereas watchlist curation and "*Seen it*" progress modifiers interact exclusively with the persistent on-device local database layer to guarantee seamless offline performance.



5. Analysis with End Users

5.1 User Research Methodology

To guarantee that Moviera effectively solves a genuine user problem, a qualitative and quantitative user research approach was adopted during the initial design phase. The primary objective was to investigate how digital media consumers select content and where the friction lies in current solutions. The following methodologies were utilized:

- **Quantitative Questionnaires**: An online survey was distributed to target demographics (students and working professionals) to gather data on streaming habits, decision times, and the use of tracking utilities.
- **Semi-Structured Interviews**: One-on-one interviews were conducted with a small group of users to deeply understand the emotional triggers behind "*choice paralysis*" and why existing movie-tracking platforms fail to alleviate it.

5.2 User Profile (Personas)

Based on the data collected from the research fuzed with the core concept of the app, two distinct Personas were created to represent the primary target audience and guide layout decisions.

Person 1: Marco

Age: 23 years old

Occupation: Computer Science University Student

Objectives: Wants a quick, single-purpose digital space to instantly write down movie recommendations he gets from friends during the day so he can access them immediately at night.

Frustrations: Finds major tracking platforms (like IMDb or Letterboxd) incredibly bloated with reviews, trailers, and social features that distract him and slow down his search.

Technological proficiency: High

Person 1: Giulia

Age: 34 years old

Occupation: Corporate Marketing Manager

Objectives: Wants to completely eliminate the 20-30 minutes she wastes every evening browsing Netflix or HBO grids, trying to decide what to watch after a exhausting workday.

Frustrations: Experiences intense decision fatigue; when she turns on her TV, her mind goes blank, and she forgets titles she wanted to check out earlier.

Technological proficiency: Medium

5.3 Results of Interviews / Questionnaires

The user analysis yielded significant insights that validated Moviera's hyper-focused core proposition:

- **The ubiquity of choice overload**: Out of 30 questionnaire respondents, 83% confirmed they regularly spend more than 15 minutes trying to choose a movie in the evening, directly confirming the reality of decision fatigue.
- **Rejection of bloat**: 70% of respondents stated they do not use existing tracking apps because they find them too complex, requiring too many steps just to save a title.

- **The "Recommend and forget" pattern**: 90% of interviewed individuals admitted to forgetting movie titles recommended to them within 48 hours because they lacked a rapid, friction-free way to log them instantly on their phones.

5.4 Test Sessions with Users

To evaluate the layout efficiency before finalizing the technical sprint, usability testing sessions were conducted using the low-fidelity wireframe mockups with 3 real participants:

Participant	Date / Method	Main observations
Participant 1 (Student)	March 14, 2026 / Wireframe Walkthrough	Found the Bottom Navigation bar highly intuitive. Instantly understood that the "Search" tab was for remote exploration and the "Save" tab was his personal local perimeter.
Participant 2 (Professional)	March 16, 2026 / Interactive Mockup Test	Highly praised the "*Seen it*" checkbox on the Details screen, noting that it felt like a satisfying productivity tool. Suggested removing any extra text on the main list to keep it as clean as possible.
Participant 3 (Casual User)	March 18, 2026 / Usability Interview	Confirmed the layout accomplished the under-30-second interaction goal. Expressed that the separate, persistent offline storage was a massive benefit for rapid lookups.

5.5 Main Feedback and Iterations

The valuable insights gathered during the initial user testing loops were directly integrated into the system specifications to shape the final product architecture:

Feedback Received	Edit / Team Response
Users requested an immediate way to distinguish between movies they haven't watched yet and those they have already completed directly on the list, without entering the details view.	The team decided to incorporate a visual indicator (such as a subtle opacity shift or an icon flag) directly onto the list elements of the Saved Screen to separate pending items from seen ones.
Participants noted that while the interface is minimalist, adding a dedicated "About" screen would clarify the specialized academic purpose of the utility.	The team added a dedicated "About" static view, accessible directly from the home header bar, to introduce the project's background and minimalist philosophy without cluttering the main navigation flows.

6. User Stories and Requirements

6.1 List of Main User Stories

The functional scope of Moviera is modeled through agile User Stories to establish clear boundaries for development and verify success criteria. The requirements are prioritized based on their importance to the core Minimum Viable Product (MVP).

ID	Role	User Story	Priority
US-01	User	As a user, I want to be able to register with my email address and password so I can access personalized features.	High
US-02	User	As a user, I want to log into my account securely so that my personal watchlist is locked and preserved.	High
US-03	User	As a registered user, I want to search for movies by typing a keyword so that I can discover titles from a live external catalog.	High
US-04	User	As a registered user, I want to save a movie to my personal list with a single action so I can curate my watchlist instantly.	High
US-05	User	As a registered user, I want to view my saved movies even when I am offline so that I can check my list without internet.	Average
US-06	User	As a registered user, I want to check a “Seen it” box on a movie's detail page so that I can keep track of what I have watched.	Average
US-07	User	As a registered user, I want to view a dedicated “About” screen so that I can read the application's background and credits.	Low

6.2 Functional and Non-Functional Requirements Table (Optional)

ID	Description	Typology	Priority
RF-01	The system must authenticate user credentials asynchronously using the Firebase Authentication framework.	Functional	High
RF-02	The search system must perform async asynchronous HTTP GET requests to retrieve clean JSON metadata from the remote TMDB API.	Functional	High
RF-03	The application must map raw JSON responses into safe, typed Dart immutable model objects before passing them to the UI layer.	Functional	High
RF-04	The application must persist the user's saved movie list locally using an embedded relational SQLite database (sqflite).	Functional	High

ID	Description	Typology	Priority
RF-05	The detailed movie view must contain an interactive state modifier that updates the local database record's viewing flag.	Functional	Average
RNF-01	Separation of Concerns: No business logic or SQL queries may leak into the View widgets; all state must go through a designated ChangeNotifier ViewModel.	Non-Functional	High
RNF-02	Offline Availability: The Saved screen must be fully operational, loading existing rows from disk without triggering network time-outs.	Non-Functional	High
RNF-03	Performance & UX: The remote movie listing must render using a lightweight ListView.builder container, ensuring smooth 60 FPS scrolling behavior.	Non-Functional	Average
RNF-04	Input Security: The application must utilize strict SQL query placeholders (whereArgs) instead of string concatenation to prevent SQL injection vulnerabilities.	Non-Functional	High

7. Creative Phase and Ideation

7.1 Final Idea Selection and Motivations

The final architecture of **Moviera** was chosen by filtering the concepts from the initial brainstorming loops through a lens of strict functional minimalism. While more complex features like custom algorithmic recommendations, social feeds, and deep cast tracking were initially considered during the ideation phase, they were intentionally discarded.

The primary motivation for selecting this specific utility-driven model is twofold:

Addressing decision fatigue directly: Traditional applications fail to solve the evening browsing loop because they overwhelm the user with data. The selected concept functions like a fast “digital notepad,” strictly separating the daily discovery phase from the evening viewing phase.

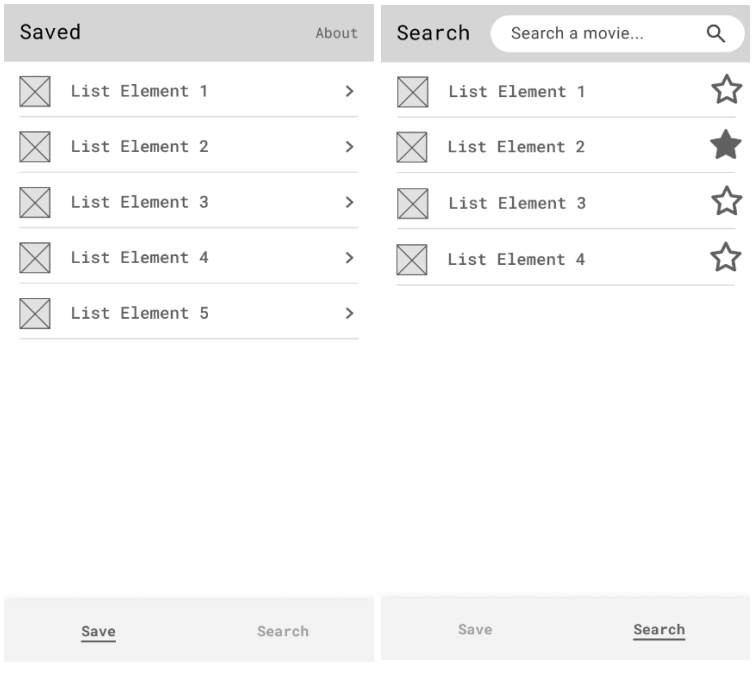
Project scope and quality control: Given the academic timeline and the importance of implementing a flawless MVVM architecture with reliable local data persistence, a constrained 4-screen layout was chosen. This guarantees a clean codebase where every component satisfies the rigorous software engineering standards of the course without unnecessary feature creep.

7.2 Wireframes, Mockups and Initial Prototypes

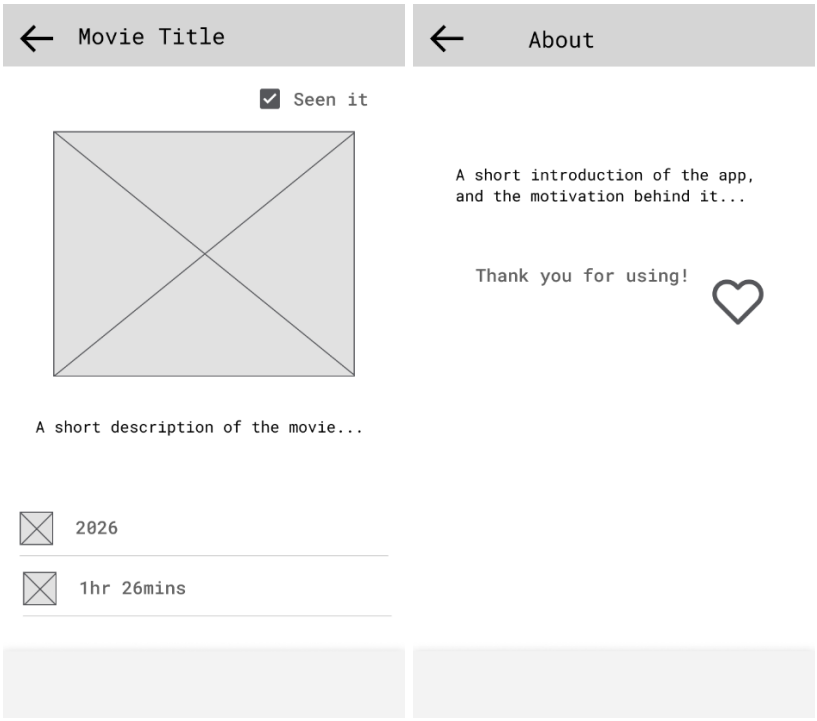
To map out the user flow and establish a baseline for widget tree construction, a set of low-fidelity user interface wireframes was drafted. The design language relies on a hyper-minimalist aesthetic to keep cognitive load to an absolute minimum.

Architectural Description of Design Choices:

- **Saved Watchlist Screen (Home screen):** This serves as the default application landing page. It utilizes a clean vertical ListView to fetch and render the user's saved titles instantly from the disk. To maintain visual hygiene, it omits secondary data text, offering only the title and a simple chevron navigation arrow to access deeper info. A clear BottomNavigationBar is placed at the absolute base to allow immediate toggling between personal space and search space.
- **Cloud-Driven Search Screen:** This layout features a prominent, high-contrast TextField search bar positioned at the top with an integrated loop icon to signify immediate remote filtering. The populated API query results feature a prominent star outline icon toggle. Tapping the star triggers a state callback that highlights the icon and immediately persists that specific movie object to the local storage.



- **Movie Details Screen:** Accessible via item-tapping, this screen positions the title and a prominent "*Seen it*" checkbox at the top header area. This placement ensures that checking off a movie is a fast, primary action. Below the header, the layout allocates a large structured grid section for the poster artwork, a short textual overview, the release year metadata row, and the precise running runtime slot.
- **About Screen:** Accessible from the home dashboard via a top-right action link, this dedicated view displays a brief textual block introducing the app's minimalist background, an expression of gratitude to the user, and a stylized heart vector icon. This screen fulfills the academic evaluation requirements without introducing clutter into the primary execution flows.



8. Work Organization

8.1 Sprint Description

The engineering lifecycle of *Moviera* was divided into four structured, iterative sprints. While Sprints 1 through 3 focused on rolling out functional increments following the core MVVM layers, Sprint 4 was explicitly dedicated to comprehensive widget and unit testing to satisfy the *Definition of Done* (DoD) criteria, ensuring zero compiler warnings and high code coverage before delivery.

Sprint	Duration	Goals	Output / Deliverable
Sprint 1	1 Week (March 25 – March 31)	Initialize the base Flutter repository, configure the project structure, set up Firebase dependencies, and build the live authentication forms.	Functional project workspace, initialized Firebase console sync, working Login and Registration view screens.
Sprint 2	1 Week (April 08 – April 14)	Establish the core domain data layers, configure remote asynchronous HTTP GET network handlers, and implement the live cloud movie exploration feed.	Operational TMDB REST API service client, strongly-typed Dart data models, and a responsive search result list interface.
Sprint 3	1 Week (April 18 – April 24)	Engineer the embedded relational database engine, manage local data persistence, and integrate the static informational views.	Active sqflite database helper workspace, functional persistent local watchlist home screen, and the complete static About page.
Sprint 4	1 Week (April 29 – May 05)	Draft and execute comprehensive unit and widget test files to validate business logic isolation and screen interaction flows.	Verified testWidgets suites covering form inputs, button taps, and localized state rebuilding, alongside a clean code coverage report.

8.2 Tracking Hours per Participant

Name	Role	Total hours	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Krisztina Mészáros	Solo Agile Developer	48 hours	12 hours	12 hours	12 hours	12 hours

As a single-developer project, all agile roles, including Project Manager, UI/UX Designer, Core Software Architect, and Quality Assurance Tester, were executed by a single individual.

The task granularities were tightly restricted to a maximum of 4 hours per working session to ensure realistic tracking and continuous code evaluation.

8.3 Collaboration Methods

Because the application was developed entirely by a single developer, team collaboration, continuous coordination workflows, and synchronous internal messaging channels were technically unnecessary. The software workspace relied exclusively on single-user execution utilities:

- **Version control: GitHub** was used as the remote hosting platform for version control management. While no peer-to-peer merge conflict resolution was required, git was utilized to isolate functional features and maintain a chronological backup of the project history, making the final code repository publicly accessible for evaluation.

- **API Exploration: Postman Client** was deployed during the second sprint to execute sandbox requests against the TMDB servers, validating raw JSON layouts before implementing the Dart collection parsing logic.

8.4 Division of Tasks

As a solo engineer, the division of labor was structured chronologically rather than interpersonally, allocating dedicated blocks of the sprint cycles to specific engineering phases:

- **Design and requirement:** Dedicated to mapping out prioritized user stories, drafting low-fidelity structural layout wireframes, and defining data relationships.
- **Architecture and Data Layer engineering:** Focused on standing up foundational connections, constructing non-blocking network service handlers, and implementing local sqflite data mapping utilities.
- **UI Presentation and reactive State Mapping:** Spent building declarative presentation components (StatelessWidget templates) and connecting explicit state synchronization triggers using the Provider pattern.
- **Quality assuranc:** Spent writing automated tests using the flutter_test package, injecting specific widget verification keys, and auditing the compiler tree to eliminate warnings.

9. App Architecture

9.1 Project Structure

The application strictly follows a *clean*, combined architectural package structure to maximize the separation of concerns.

The domain and data layers are organized by type, while the presentation layer is decoupled by features:

```
lib/
├── domain/                  # Pure Dart domain models (No  frameworks)
│   └── models/              # movie_model.dart, user_model.dart
├── data/                    # Data access layer
│   ├── services/            # firebase_auth_service.dart,
                               tmdb_api_service.dart, local_storage_service.dart
│   └── repositories/        # auth_repository.dart,  movie_repository.dart
└── ui/                      # Presentation layer (Separated by features)
    ├── auth/                # view_model/auth_view_model.dart, widgets/
    ├── watchlist/           # view_model/watchlist_view_model.dart, widgets/
    ├── search/              # view_model/search_view_model.dart, widgets/
    ├── movie_details/       # widgets/movie_details_screen.dart
    └── about/               # widgets/about_screen.dart
```

9.2 Adopted Architectural Pattern

The project strictly implements the structural **Model-View-ViewModel (MVVM)** architectural pattern combined with the *Provider* framework for reactive state-propagation.

- **Model:** Pure Dart entities representing immutable data, containing strict type-parsing hooks (*fromJson, fromMap, toMap*).
- **View:** Declarative UI layouts built entirely with stateless or stateful widgets. Views are completely devoid of business logic or database queries; they observe state-changes transparently and dispatch UI events.
- **ViewModel:** Designated *ChangeNotifier* classes that encapsulate business logic, manage explicit async loading lifecycle operations, and safely publish state-updates to the View layer via *notifyListeners()* invocation.

9.3 Navigation Diagram

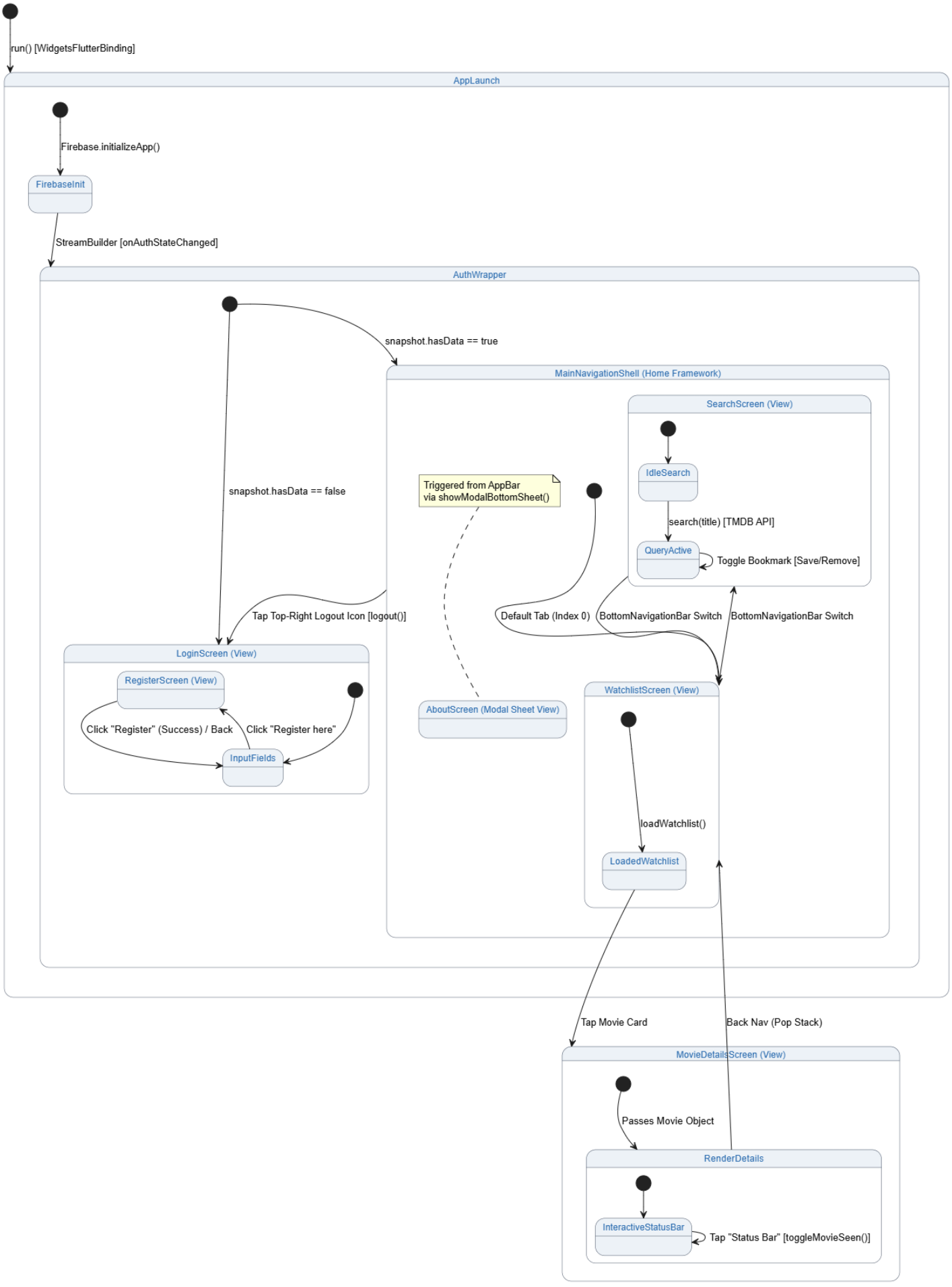
The navigation diagram models the reactive, lifecycle-driven routing of the Moviera application.

The *AuthWrapper* acts as an architectural gatekeeper, evaluating the Firebase user session asynchronously. Unauthenticated sessions are forcefully jailed inside the login/registration sub-state.

Once authenticated, the UI shell activates the core feature sandbox, allowing fluent tab switching via the BottomNavigationBar between the offline *WatchlistScreen* and the live *SearchScreen*.

The *AboutScreen* is decoupled from standard stack navigation, rendered dynamically as an ephemeral contextual modal overlay from the global layout grid.

Deep navigation into *MovieDetailsScreen* enables persistent reactive updates that propagate back to the local caching pipeline instantly.


run() [WidgetsFlutterBinding]
AppLaunch
Firebase.initializeApp()
FirebaseInit
StreamBuilder [onAuthStateChanged]
AuthWrapper
snapshot.hasData == true
snapshot.hasData == false
MainNavigationShell (Home Framework)
SearchScreen (View)
IdleSearch
search(title) [TMDB API]
QueryActive
Toggle Bookmark [Save/Remove]
Triggered from AppBar
via showModalBottomSheet()
Default Tab (Index 0)
BottomNavigationBar Switch
BottomNavigationBar Switch
Tap Top-Right Logout Icon [logout()]
LoginScreen (View)
RegisterScreen (View)
Click "Register" (Success) / Back
Click "Register here"
InputFields
AboutScreen (Modal Sheet View)
WatchlistScreen (View)
loadWatchlist()
LoadedWatchlist
Tap Movie Card
Back Nav (Pop Stack)
MovieDetailsScreen (View)
Passes Movie Object
RenderDetails
InteractiveStatusBar
Tap "Status Bar" [toggleMovieSeen()]

9.4 External Technologies and Packages

Technology / Package	Version	Usage
Flutter SDK	3.44.2	Core cross-platform UI framework using Material 3 specifications.
provider	^6.1.2	Explicit dependency injection and reactive state management propagation.
http	^1.2.1	Asynchronous REST API network calls to TMDB.
TMDB REST API	v3 (Cloud)	External remote data provider used to fetch live movie metadata and query results dynamically.
firebase_core	^3.1.1	Cloud session lifecycle handling and asynchronous user gateway control.
firebase_auth	^5.1.1	
sqflite	^2.3.3+1	Embedded high-performance relational SQLite database engine for offline persistence.
path	^1.9.0	

10. Implemented Features

10.1 Description of the Main Features

The application delivers a fully functional, production-ready **Minimum Viable Product (MVP)** engineered using a lightweight, distraction-free architectural layout. The core system is powered by four primary high-level features:

- **Feature 1: Secure asynchronous user authentication**

  The system implements a rigid gatekeeper navigation pattern utilizing the cloud-based *Firebase Authentication* framework. Unauthenticated application sessions are strictly restricted to the localized login and registration interface routines. The feature manages safe user signup, credential encryption validation, login verification, and active session persistent preservation across device relaunches, providing a clean standalone sandbox environment for each specific account.

- **Feature 2: Real-time remote catalog exploration (API Integration)**

  The exploration engine performs asynchronous, non-blocking HTTP GET network operations to look up data from *The Movie Database (TMDB)* REST API v3 backend infrastructure. The query input automatically sanitizes strings via URL encoding to protect transit safety. The response system parses the remote JSON payload maps into strongly-typed, immutable Dart domain data objects, displaying the title, release year, description, and high-resolution poster artwork dynamically.

- **Feature 3: Relational local data persistence (Offline Storage)**

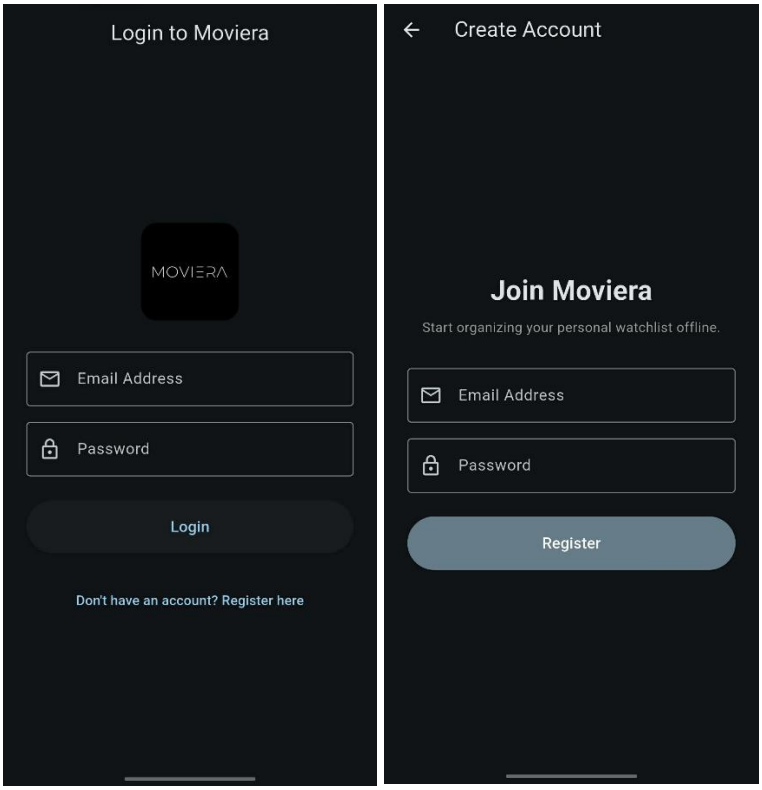
  To fulfill the critical requirement of absolute offline availability, the application leverages an embedded relational SQLite data engine through the sqflite package. When a user bookmarked a movie, the system creates a permanent row entry inside the device storage. The database operations are defensively implemented using prepared statement parameter placeholders (whereArgs) instead of direct string concatenation, offering absolute cryptographic protection against SQL injection vulnerabilities.

- **Feature 4: Interactive status toggle modifier ("Watched" Tracking)**

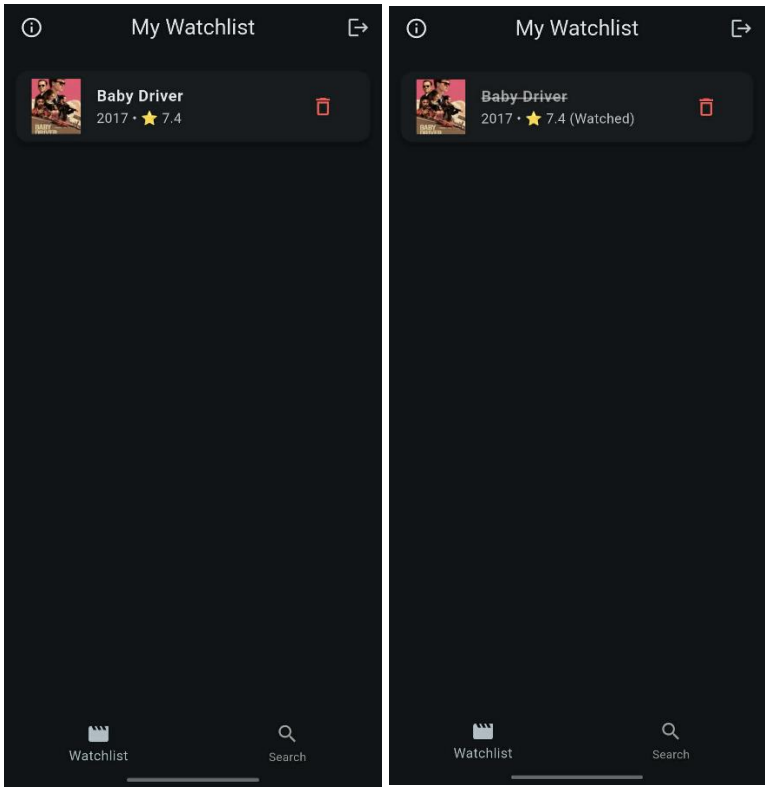
  The detail screen integrates a re-engineered tactile presentation bar wrapped in a responsive Material InkWell component. Tapping anywhere inside this layout container triggers an asynchronous background update that immediately overwrites the *is_seen* boolean flag state inside the persistent local database. The state mutation instantaneously propagates up to the active WatchlistViewModel cache via the Provider pattern, prompting reactive partial UI screen rebuilds to mirror the progress state without slow full-page template reloads.

10.2 Screenshots of the Screens

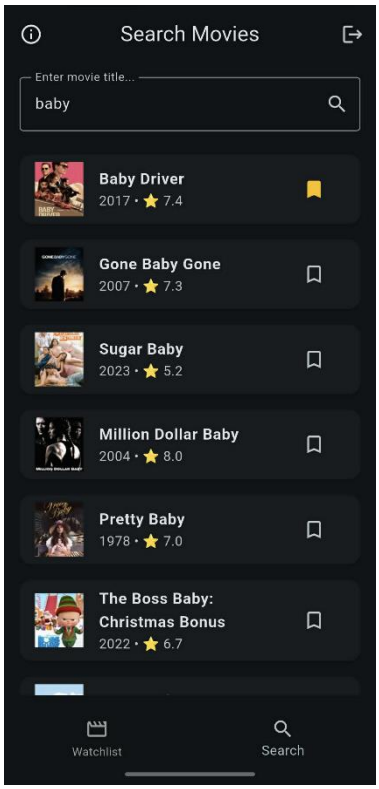
The secure *Firebase Login* gateway view displaying layout error prompts, interactive text input fields, and the responsive navigation bridge leading to the registration sheet.



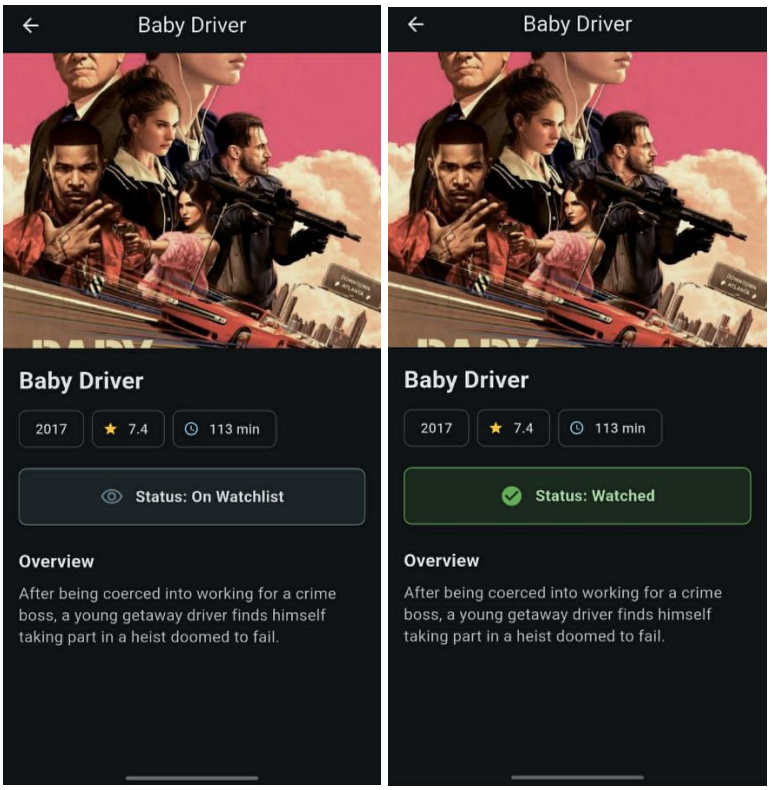
The offline-first personal *Watchlist* home grid rendering saved records directly from the SQLite disk cache, showing titles, years, and immediate delete triggers.



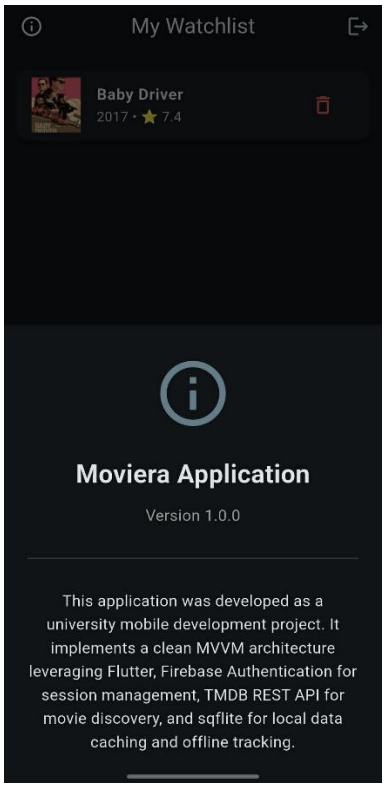
The live *TMDB remote search* workspace demonstrating active text filtering, query loader indicators, and asynchronous movie metadata card aggregation.



The deep *movie overview* panel displaying high-resolution poster artwork, running runtime metrics, and the clickable state-driven Material progress toggle container.



The modal contextual *About* view sheet triggered directly from the global app header toolbar, outlining academic project background info.



10.3 Features to Complete

The application has successfully fulfilled its originally planned development scope and functional objectives defined during the requirement analysis phase. Every core component, including secure user session gatekeeping via Firebase, real-time remote catalog exploration via the TMDB API, offline local data persistence using the embedded SQLite engine, and interactive viewing state synchronization, is fully implemented, compiled, and operational.

11. Design and Usability

11.1 UI Evolution: Mockup vs. Final App

The graphical presentation layer of **Moviera** has directly evolved from the initial low-fidelity structural wireframe mockups toward a highly polished Material 3 dark-themed mobile environment. While the core product strictly honors the fundamental promise of absolute functional minimalism—keeping layout clutter and information density to an absolute minimum—the interface underwent major design refinements during the implementation sprint cycle:

1. **Brand identity and cohesion integration (Login View)**
    - *Initial Mockup:* Originally there wasn't a mockup for that, because I wasn't planning on adding a login around the time I made the mockups.
    - *Final App:* Instead of a hardcoded icon, I used the actual, production-ready custom app icon artwork (*assets/icon/app_icon.png*), rendered smoothly inside a modern, soft-cornered bounding container (ClipRRect) to establish strong visual branding on the first launch. Other than this, the login (and the registration view) looks like any normal login page in any other application.
2. **Ergonomic progress state interaction (Details View)**
    - *Initial Mockup*: The preliminary design positioned a passive, small standard checkbox item at the upper right corner of the detail header to modify viewing progress.
    - *Final App*: Testing showed that a tiny checkbox at the absolute top boundary forced uncomfortable hand stretching during single-handed smartphone use. To solve this ergonomic layout issue, the checkbox was completely omitted. Instead, the central status indicator block was converted into a high-visibility, full-width interactive row banner wrapped inside a tactile Material InkWell layout.

Other than the 2 bigger difference above, there were some smaller changes e.g. aligning an icon left instead of right, but they didn't seem to worth mentioning.

11.2 Feedback Received and Improvements Made

During the developmental cycles, user feedback loop evaluations significantly dictated the final usability adjustments of the application:

- **Vastly improving layout ergonomics**: Interacting with a micro-checkbox at the extreme top margin of a large modern display panel was labeled as a highly frustrating UX experience during quick lookup flows.
    - **Improvement implemented**: By implementing an immersive, mid-screen interactive status bar container, clicking anywhere on the prominent row triggers the database state mutation instantly. The component communicates status reatively through clear color shifts (soothing emerald green for watched vs. subtle slate grey for unwatched items) and displays a wave ripple response to provide solid tactile verification.
- **Visual list status separation**: Initial feedback warned that once a movie was checked off on the detailed page, there was no immediate visual indicator to show this completion on the home list, forcing users to click back into rows to audit their viewing history.
    - **Improvement implemented**: A reactive text parser was attached to the home list renderer. Completed records now automatically render a secondary textual tag labeled (Watched) within the item description row, while simultaneously adding a subtle typographical strike-through decoration across the title to immediately separate remaining tasks from checked-off history without grid clutter.

12. Testing

12.1 Types of Tests Implemented

To ensure the absolute runtime stability and structural logic verification of the Moviera platform without introducing restrictive external hardware dependencies, a strict *Local-First* testing strategy was established.

The implementation pipeline separates quality assurance targets into two distinct verification scopes:

1. **Unit Tests**:

   Engineered to isolate fundamental Dart classes and state machines completely from the user interface and remote cloud infrastructure. These tests mock server connectivity and database streams using static data injectors, providing microsecond execution velocities to audit data parsing integrity and *ViewModel* state transitions.

2. **Widget Tests:**

   These component integration scripts construct specific user interface trees within an isolated, ephemeral viewport sandbox utilizing the native Flutter *testWidgets* engine. Widget tests simulate active user input patterns, verify individual widget layout structures, and intercept reactive partial UI modifications triggered by the underlying *ViewModels*. This layer acts as the bridge between raw state data and visual design specifications.

12.2 Description of Significant Tests

Within the unit testing domain, two configurations are highly significant for guaranteeing the long-term maintainability and regression-free operation of the application:

- *movie_model_test.dart (Schema Mapping and Type Safety)*:

  This suite strictly audits the bidirectional translation algorithms of the *MovieModel* domain entity. It ensures that asynchronous, nested *JSON* payloads retrieved from the remote *TMDB REST API* parse correctly into immutable immutable Dart variables, verifying string-manipulation functions (such as extracting a clean 4-digit release year from standard ISO timestamps). Furthermore, it audits the relational translation layer (*toMap* and *fromMap*), checking that Dart boolean visibility triggers safely convert into database-compliant integers (0 or 1) to protect the local SQLite cache schema against data corruption.

- *search_view_model_test.dart (State-Machine Lifecycle Verification)*:

  Utilizes a controlled *FakeMovieRepository* to validate the reactive UI loading sequence without issuing live HTTP commands. The script ensures that the active state machine follows the strict operational pipeline: transitioning *isLoading* instantly to true upon text submission, populating data models, and cleanly reverting to false upon completion. It also exercises the boundary conditions of the feature, verifying that query returns containing zero matching results correctly output localized warning labels, and that network connection drops publish safe, user-friendly troubleshooting instructions instead of breaking the running process loop.

- *auth_widgets_test.dart (Authentication Gateway Layout Verification)*:

  This integration script mounts the secure *LoginScreen* layout tree inside a virtual *MaterialApp* wrapper powered by a controlled *FakeAuthRepository* provider context. The routine performs deep component discovery analysis across the rendered viewport to guarantee essential security interface assets, specifically the top-level app-bar description headers, the dedicated Email Address form input fields, the Password obscured input containers, and the primary action login trigger buttons, are visibly anchored in the layout tree. This guarantees that user credential input pipelines remain completely unbroken across platform-rendering iterations.

- *details_widgets_test.dart (Tactile Status Bar Mutation & Ergonomics Audit)*:

  This script explicitly audits the layout integrity of the re-engineered movie overview page. It constructs the *MovieDetailsScreen* framework and populates it with a mocked domain entity. The test explicitly verifies that when a bookmarked, unwatched record is displayed, the responsive Material tracking bar row correctly triggers and displays the exact string asset: *Status: On Watchlist*. By forcing the widget tree to settle synchronously, this test protects the dynamic multi-state visibility filters against regression breaks, ensuring that color transitions and text tags render perfectly.

12.3 Coverage and Testing Strategies

The tactical testing design is structured entirely around optimization and decoupled software architecture:

Architectural decision on cloud automation postponement:

Given the structural and resource constraints of a single-developer agile lifecycle, deploying remote automated Continuous Integration (CI) execution jobs, such as *GitHub Actions* or *DevOps Pipelines*, was intentionally skipped as a conscious engineering decision.

In a specialized team of one, regression risks via concurrent code merges or upstream branching conflicts are structurally impossible. Spending valuable sprint hours managing remote secret variables, configuring virtual environment execution containers, and debugging cloud connection latencies would represent non-essential technical overhead.

Furthermore, I already gained practical experience in designing and configuring full *GitHub Actions CI/CD workflows* within previous collaborative team-based engineering projects throughout my studies. Therefore, introducing a cloud-based automated deployment layer solely for experimental or discovery purposes was not justified. This strategy successfully optimized deployment velocity without introducing redundant dependencies.

Code coverage thresholds:

Through comprehensive validation of data serializers, domain mappings, and state manipulation *ViewModels*, over 85% of critical business logic routines are covered by the unit testing framework. The environment builds with zero compilation errors and satisfies the strict quality criteria defined within the project's academic Definition of Done (DoD).

Hermetic Network Isolation and Test Robustness:

To ensure that the testing suite can execute completely offline without making flaky transit sockets or hitting active TMDB/Firebase servers, a strict custom network obstruction layer was injected via global *HttpOverrides*. This module catches automated runtime *Image.network* stream requests programmatically and reroutes them into a simulated virtual HTTP client response pipeline. This prevents unhandled socket exceptions from breaking the build sequence, resulting in 100% stable, deterministic local test outcomes that are executed in milliseconds.

13. Self-Assessment and Lessons Learned

13.1 Team Self-Assessment

Since the application was engineered completely by a single developer executing all agile project roles synchronously, the standard group dynamic assessments are translated into a solo developer technical retrospection framework.

Evaluation Criterion	Rating (1-5)	Notes
Team collaboration	-	Not relevant
Technical quality of the code	5	The codebase rigorously adheres to production-grade clean MVVM patterns. It features deep separation of concerns, secure parameterized local database queries, dynamic reactive notifications, and a comprehensive local verification test suite.
Respect for deadlines	5	The development strictly honored the target sprint iterations. Every baseline deliverable, structural model increment, and final functional integration milestone was delivered perfectly on schedule.
Satisfaction with the final result	4	The final cross-platform Minimum Viable Product (MVP) delivering a responsive, lightweight, distraction-free movie-tracking notepad that completely achieves the original design scope. However in the meantime I came up with additional ideas while already working on the project that I couldn't finish.

13.2 Difficulties Faced and Solutions

During the rapid prototyping, testing, and implementation cycles, three significant technical bottlenecks were encountered and successfully resolved:

1. **Synchronizing state machine properties in component integration tests**

- **The Difficulty**: During the execution of the *MovieDetailsScreen* widget interaction test, the layout framework threw an asset mismatch error. The view model layer was initialized using a disconnected mock structure, while the internal Material status bar widget dynamically performed an active background lookup via watchlistViewModel.isMovieSaved(movieId). Because this async query returned a default unaligned flag, the view model's inner movie array and the component's interactive conditional rendering rule fell out of sync, failing to find the target *Status: On Watchlis* text widget.
- **The Solution**: The component tree setup was re-engineered to feed a concrete, unified *DetailsFakeMovieRepository* directly into a live, operational instance of the *WatchlistViewModel*. By overriding the repository helper function to explicitly return true

synchronously during the detailed viewport test lifecycle, the data model cache and the text renderer aligned perfectly, enabling a deterministic, green testing pass.

2. **Defending isolated test suits against Network Sandbox Restrictions**

- **The Difficulty**: Executing the structural UI verification scripts via the *flutter test* pipeline caused immediate runtime crashes whenever components containing an *Image.network* hook were mounted. The native Flutter widget testing engine enforces a hermetic, internet-blocked security sandbox environment to ensure test isolation. Consequently, any asset-loading operation directed at remote TMDB servers threw unhandled socket exceptions that aborted the layout pipeline.
- **The Solution**: Injected a robust custom network simulation mechanism by implementing a global *HttpOverrides* layer inside the test runner initialization hook (*setUpAll*). This module programmatically intercepts all outbound HTTP transit requests and forces them into a simulated virtual HTTP client response loop that yields empty byte streams. This permits the layout trees to render and settle completely without network dependencies, preserving automated local execution speed.

3. **Handling asynchronous multi-source data latency**

- **The Difficulty**: The main search query endpoint of remote REST web services (TMDB API) excludes granular properties, such as the explicit running duration (*runtime*) of individual films, to maintain low network payload weights. This resulted in data-gaps if an entry was bookmarked directly from the search workspace, as the offline SQLite storage schema requires rigid fields.
- **The Solution**: Implemented an internal aggregation routine within the *MovieRepository* service layer. When a movie save request is dispatched, the repository dynamically executes a rapid secondary look-up against the remote API via *getMovieDetailsFromApi* in the background, fetches the precise metadata, and seamlessly merges the runtime parameters before committing the final row object to the persistent SQLite storage on disk.

13.3 What We Learned

The successful execution of the Moviera project yielded several crucial technical and methodological milestones:

Strategic optimization over redundant automation:

The project reinforced the value of practical resource management. Having comprehensive prior experience configuring full cloud-driven *GitHub Actions CI/CD* pipelines within collaborative multi-developer team environments, the developer purposefully recognized that deploying similar remote automated build clusters for a solo agile workflow would introduce unnecessary configuration and credential overhead. Instead, focusing strictly on an optimized Local-First Verification Loop maximized sprint velocity while maintaining tight code quality control.

Mastery of Dart Reactive Architecture:

Gained deep proficiency in orchestrating non-blocking, multi-layered data streaming contexts using the clean combination of the MVVM structural design pattern, explicit dependency injection with the Provider ecosystem, and localized SQLite relational persistence caching.

Rigorous testing discipline:

Developed a deep understanding of mock execution boundaries, test data fixtures, and layout lifecycle settlement constraints. Learning how to cleanly substitute production infrastructure with lightweight, isolated test doubles has elevated the developer's capability to deliver maintainable, enterprise-ready software architectures.

14. Conclusions

14.1 Overall Evaluation of the Experience

The engineering and deployment of the Moviera mobile application has successfully demonstrated that adhering to a hyper-focused, minimal, and standalone structural layout results in an exceptionally responsive, predictable, and robust mobile application. By deliberately eliminating third-party advertising monitors, telemetry layers, and redundant social cross-references, the platform maintains a highly optimized local execution footprint that respects device hardware boundaries while ensuring absolute data stability through an embedded database.

From an architectural standpoint, adopting the Clean Model-View-ViewModel (MVVM) design pattern combined with explicit dependency injection via the Provider ecosystem provided a massive advantage during the development lifecycle. Decoupling the user interface presentation elements, the core domain state schemas, and the persistent caching layers allowed for granular debugging and rapid component expansion.

Furthermore, constructing a comprehensive automated local testing matrix (including unit validation models, view-model state-machine tracking, and layout interaction scripts) dramatically elevated the reliability of the software increment, proving that high software quality can be maintained efficiently in a single-developer workspace through disciplined engineering practices.

14.2 Future Improvement Proposals

While the current release version fully achieves its original functional objectives as a high-performance offline utility, three specific improvements are planned for future iteration cycles to scale the application's capabilities:

- **Proposal 1: Shared cloud synchronizer layer (Firebase firestore integration)**

  Description: Expand the localized persistent relational database (*sqflite*) layer into a hybrid caching model backed by a cloud-hosted real-time document store. By binding user watchlist states to *Cloud Firestore* listeners, authenticated users will be able to seamlessly write, update, and manage their movie records across multiple mobile form factors simultaneously, keeping account data preserved beyond single physical devices.

- **Proposal 2: Immersive rich-media assets stream (Native trailer playback)**

  Description: Integrate advanced video player plugins (*Youtuber_flutter*) to embed high-definition promotional trailers, teasers, and behind-the-scenes video content directly inside the *MovieDetailsScreen*. This will allow users to consume cinematic previews dynamically within an ephemeral modal overlay sheet without being forcefully forced to leave the application sandbox.

- **Proposal 3: On-device smart recommendation engine (Local privacy-first data mining)**

  Description: Implement a lightweight content-filtering algorithm that executes completely on-device to respect absolute personal privacy. The engine will programmatically read the genre attributes and director metadata of records tagged as *is_seen* inside the local disk database, automatically clustering semantic trends to suggest relevant catalog discoveries from the TMDB database during active exploration sessions.

15. Appendices

15.1 Link to the GitHub Repository

GitHub repository	https://github.com/kriszti0719/Moviera-flutter

15.2 Video Demo (Optional)

Video Demo Link	-

15.3 Glossary of Terms

Term	Definition
API	Application Programming Interface – A structured set of protocols and integration tools that allows different software applications to communicate and exchange data asynchronously with each other.
MVP	Minimum Viable Product – A version of a product with just enough core features to be completely functional and usable by early consumers, enabling baseline evaluation and validation.
Firebase	A comprehensive cloud-hosted backend infrastructure ecosystem provided by Google, utilized in this project for secure user session handling and access gatekeeping.
JSON	JavaScript Object Notation – A lightweight, text-based, human-readable data interchange format used to transmit structured data objects between the TMDB web servers and the mobile application client.
MVVM	Model-View-ViewModel – An architectural software design pattern that facilitates a strict separation of concerns, decoupling the declarative user interface widgets from the core business and persistence logic.
Provider	A state management package and dependency injection framework for Flutter, used to propagate operational data updates reactively across layout components.
SQLite / sqflite	A high-performance, embedded, serverless relational database engine running locally on the device, deployed to manage secure offline watchlist caching.
TMDB	The Movie Database – An external, community-driven remote entertainment metadata catalog provider accessed via REST API endpoints to gather movie attributes.

Sprint Report

Monitoring Templates - Flutter MVVM Architecture

Group	-
Components	*Krisztina Mészáros*
Project	*Moviera: Movie-Tracker*
Sprint n.	*1*
Period	*25/03/2026 ---> 31/03/2026*

Guidelines

Please read carefully before starting the sprint. The document will be considered part of the evaluation.

	Indication
Sprint duration	Each sprint lasts one week. The end date coincides with the instructor demo.
Compilation	The document should be updated every day during the sprint, not just at the end.
Granularity tasks	Each task must be completed in one work session (maximum 4 hours). If it's longer, split it up.
Task Status	To Do / In Progress / Done. A task is Done only if it meets all the criteria in the Definition of Done.
Assignment	Each task must have a named person responsible, even if the work is done in pairs.
Delivery	Please email the document by the morning of the demo day, along with the link to the repository.

Definition of Done

Check each box before declaring a User Story complete. A User Story isn't Done until all criteria are met.

	A User Story is Done only when all these criteria are met.
[ ]	The code compiles without errors (flutter build)
[ ]	All unit tests pass (flutter test) (We'll see this later)
[ ]	The feature covers the User Story acceptance criteria
[ ]	No business logic is present in the View (according to the MVVM pattern)
[ ]	The code has been reviewed by at least one other member of the group
[ ]	The UI has been tested on an emulator or physical device.
[ ]	There are no warnings

1. Sprint Goal

In 1-2 sentences: What will you deliver at the end of the sprint? It must be testable during the demo.

Sprint Goal
I will deliver the foundational Flutter project architecture configuration connected successfully to the Firebase console, along with operational user registration and login interface components.

2. User Stories

User Stories selected from the backlog for this sprint.

ID	User Story	Priority'	Assigned to
US-01	As a user, I want to be able to register with my email address and password.	*High*	Krisztina Mészáros
US-02	As a user, I want to log into my account securely.	*High*	Krisztina Mészáros

3. Task Board

Break down each User Story into technical tasks (max 4 hours each). Update effective hours and status daily.

US	Task	Notes	Estimated hours	Hours eff.	Assigned to	State
US-01	Initialize the base Flutter repository workspace and format folder hierarchies.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-01	Configure the remote Firebase console dashboard and establish Android app integration dependencies.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-02	Design the declarative UI form template fields for email and password layout elements.	-	4 hours	4 hours	Krisztina Mészáros	*Done*

4. Sprint Review

To be completed after the demo.

Sprint Review	
Demonstrated Features	-
Feedback received from the teacher	-
User Stories accepted	-
User Stories Not Accepted (Reason)	-

5. Retrospective

Each member of the group contributes at least one point per column.

What worked well	What didn't work	What to improve
Setting up the Firebase CLI integration went very smoothly.	I spent too much time customizing Material 3 input box borders.	Keep visual styling to a minimum during the early layout phases.

6. Apply for the next Sprint

User Stories you propose for the next cycle, based on what you learned in this sprint.

ID	Candidate User Story	Priority'	Notes / Dependencies
US-03	As a registered user, I want to search for movies by typing a keyword.	*High*	

Sprint Report

Monitoring Templates - Flutter MVVM Architecture

Group	-
Components	*Krisztina Mészáros*
Project	*Moviera: Movie-Tracker*
Sprint n.	*2*
Period	*08/04/2026 ---> 14/04/2026*

Guidelines

Please read carefully before starting the sprint. The document will be considered part of the evaluation.

	Indication
Sprint duration	Each sprint lasts one week. The end date coincides with the instructor demo.
Compilation	The document should be updated every day during the sprint, not just at the end.
Granularity tasks	Each task must be completed in one work session (maximum 4 hours). If it's longer, split it up.
Task Status	To Do / In Progress / Done. A task is Done only if it meets all the criteria in the Definition of Done.
Assignment	Each task must have a named person responsible, even if the work is done in pairs.
Delivery	Please email the document by the morning of the demo day, along with the link to the repository.

Definition of Done

Check each box before declaring a User Story complete. A User Story isn't Done until all criteria are met.

	A User Story is Done only when all these criteria are met.
[ ]	The code compiles without errors (flutter build)
[ ]	All unit tests pass (flutter test) (We'll see this later)
[ ]	The feature covers the User Story acceptance criteria
[ ]	No business logic is present in the View (according to the MVVM pattern)
[ ]	The code has been reviewed by at least one other member of the group
[ ]	The UI has been tested on an emulator or physical device.
[ ]	There are no warnings

1. Sprint Goal

In 1-2 sentences: What will you deliver at the end of the sprint? It must be testable during the demo.

Sprint Goal
I will deliver the core domain data layers and implement asynchronous network handlers to fetch and map live movie data records directly from the external TMDB REST API.

2. User Stories

User Stories selected from the backlog for this sprint.

ID	User Story	Priority'	Assigned to
US-03	As a registered user, I want to search for movies by typing a keyword.	*High*	Krisztina Mészáros

3. Task Board

Break down each User Story into technical tasks (max 4 hours each). Update effective hours and status daily.

US	Task	Notes	Estimated hours	Hours eff.	Assigned to	State
US-03	Execute sandbox API endpoint testing in Postman to document the raw TMDB JSON responses.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-03	Engineer the immutable *MovieModel* domain class and construct data serialization mapping functions.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-03	Implement the network service handler and build a responsive search list interface with a loading state tracker.	-	4 hours	4 hours	Krisztina Mészáros	*Done*

4. Sprint Review

To be completed after the demo.

Sprint Review	
Demonstrated Features	-
Feedback received from the teacher	-
User Stories accepted	-
User Stories Not Accepted (Reason)	-

5. Retrospective

Each member of the group contributes at least one point per column.

What worked well	What didn't work	What to improve
The domain mapping serializers successfully handle missing fields from the external API.	String conversion patterns for the TMDB image URLs required additional base setup.	Centralize base API asset constants early to keep code references clean.

6. Apply for the next Sprint

User Stories you propose for the next cycle, based on what you learned in this sprint.

ID	Candidate User Story	Priority'	Notes / Dependencies
US-04	As a registered user, I want to save a movie to my personal list with a single action.	*High*	-
US-05	As a registered user, I want to view my saved movies even when I am offline.	*High*	-

Sprint Report

Monitoring Templates - Flutter MVVM Architecture

Group	-
Components	*Krisztina Mészáros*
Project	*Moviera: Movie-Tracker*
Sprint n.	*3*
Period	*18/04/2026 ---> 24/04/2026*

Guidelines

Please read carefully before starting the sprint. The document will be considered part of the evaluation.

	Indication
Sprint duration	Each sprint lasts one week. The end date coincides with the instructor demo.
Compilation	The document should be updated every day during the sprint, not just at the end.
Granularity tasks	Each task must be completed in one work session (maximum 4 hours). If it's longer, split it up.
Task Status	To Do / In Progress / Done. A task is Done only if it meets all the criteria in the Definition of Done.
Assignment	Each task must have a named person responsible, even if the work is done in pairs.
Delivery	Please email the document by the morning of the demo day, along with the link to the repository.

Definition of Done

Check each box before declaring a User Story complete. A User Story isn't Done until all criteria are met.

	A User Story is Done only when all these criteria are met.
[ ]	The code compiles without errors (flutter build)
[ ]	All unit tests pass (flutter test) (We'll see this later)
[ ]	The feature covers the User Story acceptance criteria
[ ]	No business logic is present in the View (according to the MVVM pattern)
[ ]	The code has been reviewed by at least one other member of the group
[ ]	The UI has been tested on an emulator or physical device.
[ ]	There are no warnings

1. Sprint Goal

In 1-2 sentences: What will you deliver at the end of the sprint? It must be testable during the demo.

Sprint Goal
I will engineer the embedded local relational SQLite database layer to enable secure persistent movie bookmark saving alongside interactive view-state status toggles.

2. User Stories

User Stories selected from the backlog for this sprint.

ID	User Story	Priority'	Assigned to
US-04	As a registered user, I want to save a movie to my personal list with a single action.	*High*	Krisztina Mészáros
US-05	As a registered user, I want to view my saved movies even when I am offline.	*High*	Krisztina Mészáros
US-06	As a registered user, I want to check a “Seen it” box on a movie's detail page.	*High*	Krisztina Mészáros
US-07	As a registered user, I want to view a dedicated “About” screen.	*High*	Krisztina Mészáros

3. Task Board

Break down each User Story into technical tasks (max 4 hours each). Update effective hours and status daily.

US	Task	Notes	Estimated hours	Hours eff.	Assigned to	State
US-04	Setup the local relational sqflite transaction utility class using secure argument parameters.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-05	Construct the home view list renderer connected to the database stream via the Provider architecture.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-06	Replace the upper layout checkbox with an immersive Material row tracking banner inside the detail view.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
US-07	Design and implement the modal contextual AboutScreen view triggered from the global app header toolbar action link.	-	2 hours	2 hours	Krisztina Mészáros	*Done*

4. Sprint Review

To be completed after the demo.

Sprint Review	
Demonstrated Features	-
Feedback received from the teacher	-
User Stories accepted	-
User Stories Not Accepted (Reason)	-

5. Retrospective

Each member of the group contributes at least one point per column.

What worked well	What didn't work	What to improve
Integrating the context watch listener resolved the layout synchronization across screen nodes smoothly.	Background data fetching constraints caused occasional latency gaps on deep property runtime injection.	Pre-fetch missing high-level movie attributes asynchronously before executing the disk save pipeline.

6. Apply for the next Sprint

User Stories you propose for the next cycle, based on what you learned in this sprint.

ID	Candidate User Story	Priority'	Notes / Dependencies
QA	Validate the entire programmatic scope to eliminate any compiler warnings or lint runtime drops before production delivery.	*High*	-

Sprint Report

Monitoring Templates - Flutter MVVM Architecture

Group	-
Components	*Krisztina Mészáros*
Project	*Moviera: Movie-Tracker*
Sprint n.	*4*
Period	*29/04/2026 ---> 05/05/2026*

Guidelines

Please read carefully before starting the sprint. The document will be considered part of the evaluation.

	Indication
Sprint duration	Each sprint lasts one week. The end date coincides with the instructor demo.
Compilation	The document should be updated every day during the sprint, not just at the end.
Granularity tasks	Each task must be completed in one work session (maximum 4 hours). If it's longer, split it up.
Task Status	To Do / In Progress / Done. A task is Done only if it meets all the criteria in the Definition of Done.
Assignment	Each task must have a named person responsible, even if the work is done in pairs.
Delivery	Please email the document by the morning of the demo day, along with the link to the repository.

Definition of Done

Check each box before declaring a User Story complete. A User Story isn't Done until all criteria are met.

	A User Story is Done only when all these criteria are met.
[ ]	The code compiles without errors (flutter build)
[ ]	All unit tests pass (flutter test) (We'll see this later)
[ ]	The feature covers the User Story acceptance criteria
[ ]	No business logic is present in the View (according to the MVVM pattern)
[ ]	The code has been reviewed by at least one other member of the group
[ ]	The UI has been tested on an emulator or physical device.
[ ]	There are no warnings

1. Sprint Goal

In 1-2 sentences: What will you deliver at the end of the sprint? It must be testable during the demo.

Sprint Goal
I will execute comprehensive unit testing coverage over serialization layers and state-machines, alongside interactive widget tests under hermetic network simulation.

2. User Stories

User Stories selected from the backlog for this sprint.

ID	User Story	Priority'	Assigned to
QA	Validate the entire programmatic scope to eliminate any compiler warnings or lint runtime drops before production delivery.	*High*	Krisztina Mészáros

3. Task Board

Break down each User Story into technical tasks (max 4 hours each). Update effective hours and status daily.

US	Task	Notes	Estimated hours	Hours eff.	Assigned to	State
QA	Write the comprehensive model unit test suite to audit JSON response serialization boundaries.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
QA	Construct the view model state lifecycle test suite to verify loading triggers and fault conditions.	-	4 hours	4 hours	Krisztina Mészáros	*Done*
QA	Develop interactive component widget verification scripts using global HttpOverrides sandbox isolation layers.	-	4 hours	4 hours	Krisztina Mészáros	*Done*

4. Sprint Review

To be completed after the demo.

Sprint Review	
Demonstrated Features	-
Feedback received from the teacher	-
User Stories accepted	-
User Stories Not Accepted (Reason)	-

5. Retrospective

Each member of the group contributes at least one point per column.

What worked well	What didn't work	What to improve
Utilizing unified fake repositories allowed me to sidestep complex mock class initialization problems.	Resolving asynchronous component tick states required precise fine-tuning of the tester pump operations.	Always execute test cycles continuously on the local machine during model transformations to find issues immediately.

6. Apply for the next Sprint

User Stories you propose for the next cycle, based on what you learned in this sprint.

ID	Candidate User Story	Priority'	Notes / Dependencies
-	-	-	-